

	Price	Mileage	EngineV	Year
count	4173.000000	4345.000000	4195.000000	4345.000000
mean	19418.746935	161.237284	2.790734	2006.550058
std	25584.242620	105.705797	5.066437	6.719097
min	600.000000	0.000000	0.600000	1969.000000
25%	6999.000000	86.000000	1.800000	2003.000000
50%	11500.000000	155.000000	2.200000	2008.000000
75%	21700.000000	230.000000	3.000000	2012.000000
max	300000.000000	980.000000	99.990000	2016.000000

```
df.isnull().sum()
```

	0
Brand	0
Price	172
Body	0
Mileage	0
EngineV	150
Engine Type	0
Registration	0
Year	0
Model	0

dtype: int64

▼ Step 4 — Handle Missing Values

Why?

Machine learning models cannot work with empty (NaN) values.

We have missing values in `Price` and `EngineV`.

Solution: Drop rows where Price or EngineV is missing.

💡 `dropna()` removes rows that have missing values in the specified columns.

```
df.dropna(subset=['Price', 'EngineV'], inplace=True)
```

```
df.isnull().sum()
```

	0
Brand	0
Price	0
Body	0
Mileage	0
EngineV	0
Engine Type	0
Registration	0
Year	0
Model	0

dtype: int64

▼ Step 5 — Remove Outliers from EngineV

Why?

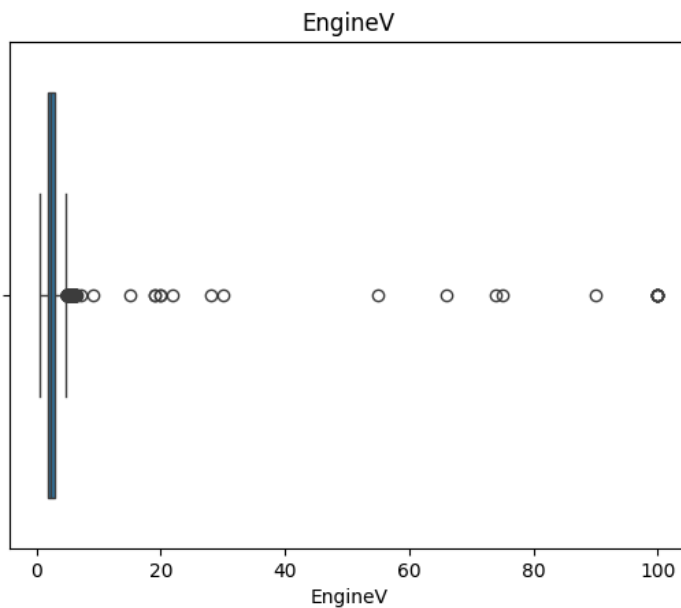
Look at the max value of `EngineV` — it shows **99.99!**
No car has a 99-litre engine. These are data entry mistakes.

Outliers confuse the model and give wrong predictions.
We keep only realistic engine sizes (≤ 10 litres).

💡 Always ask: *"Does this value make real-world sense?"*

```
import seaborn as sns
```

```
sns.boxplot(x=df['EngineV'])  
plt.title('EngineV')  
plt.show()
```

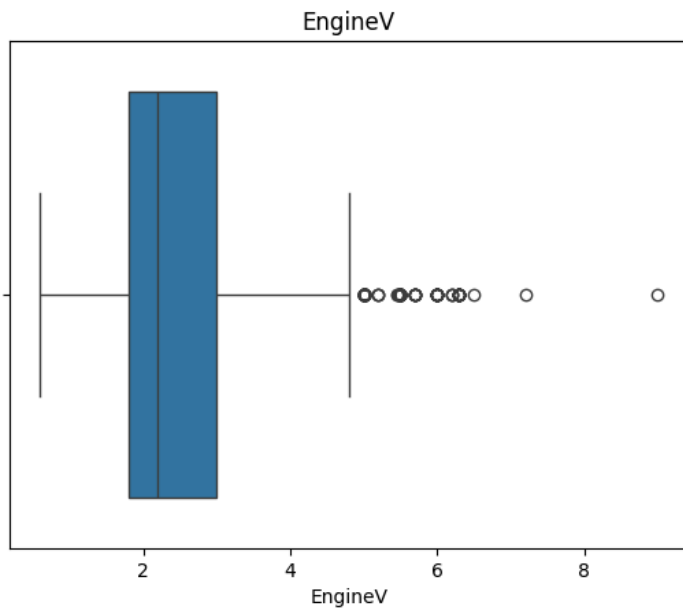


```
df = df[df['EngineV'] <= 10]
```

```
df.head()
```

	Brand	Price	Body	Mileage	EngineV	Engine Type	Registration	Year	Model
0	BMW	4200.0	sedan	277	2.0	Petrol	yes	1991	320
1	Mercedes-Benz	7900.0	van	427	2.9	Diesel	yes	1999	Sprinter 212
2	Mercedes-Benz	13300.0	sedan	358	5.0	Gas	yes	2003	S 500
3	Audi	23000.0	crossover	240	4.2	Petrol	yes	2007	Q7
4	Toyota	18300.0	crossover	120	2.0	Petrol	yes	2011	Rav 4

```
sns.boxplot(x=df['EngineV'])  
plt.title('EngineV')  
plt.show()
```



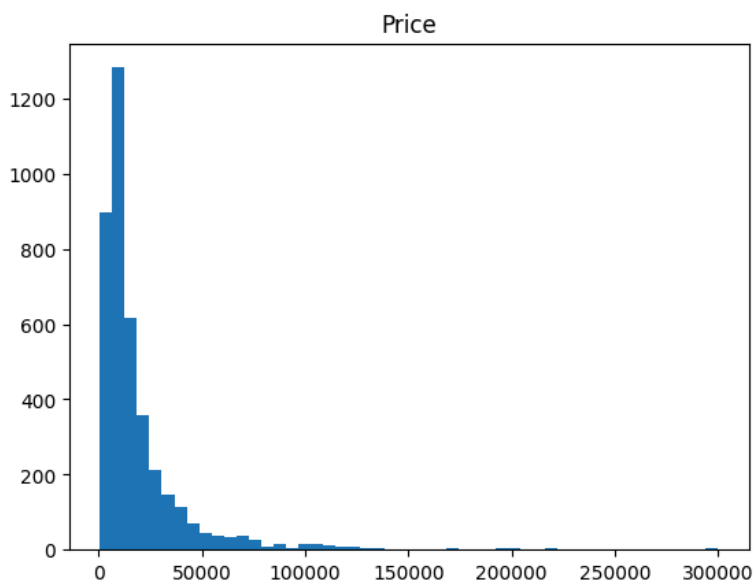
✓ Step 6 — Check Distribution of Target Variable (Price)

Why?

Linear Regression works best when the target variable is **normally distributed** (bell-shaped).

Let's plot `Price` and see its shape.

```
plt.hist(df['Price'], bins=50)
plt.title('Price')
plt.show()
```



✓ Step 7 — Apply Log Transformation

Why?

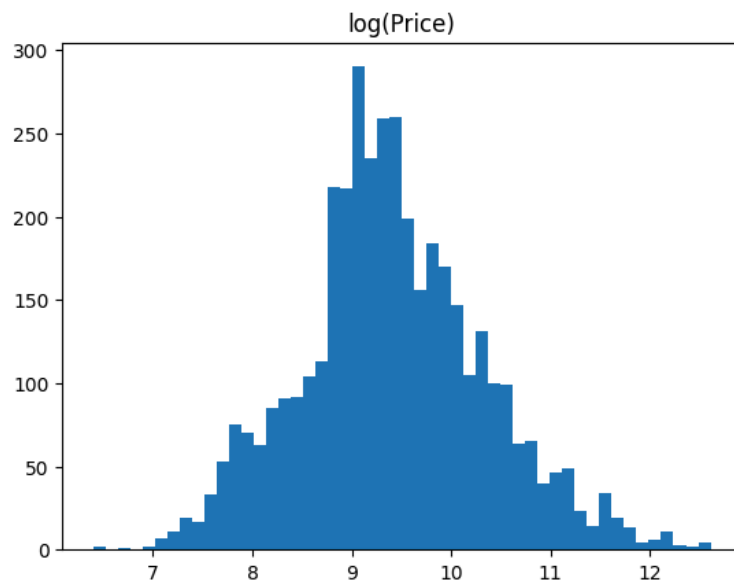
The raw `Price` is heavily right-skewed (skewness = 4.45).

After `log(Price)`, skewness drops to ~0.24 — almost normal!

This helps our model make much better predictions.

⚠ We also drop the `Model` column — it has 312 unique values (too many for simple encoding).

```
plt.hist(np.log(df['Price']), bins=50)
plt.title('log(Price)')
plt.show()
```



```
df['Price'].skew()      # ≈ 4.45 → very right-skewed
np.log(df['Price']).skew() # ≈ 0.24 → close to symmetric
```

```
np.float64(0.24414168416167548)
```

```
df['Price_log'] = np.log(df['Price'])
```

and then build your regression model to predict `Price_log` instead of raw `Price`. Later, if you want the predicted actual price, you exponentiate:

```
df['Model'].value_counts()
```

	count
Model	
E-Class	188
Vito	156
A6	156
Kangoo	132
Camry	127
...	...
Sandero StepWay	1
CLA 220	1
545	1
ML 550	1
Scion	1

305 rows × 1 columns

dtype: int64

```
df.drop('Model', axis=1, inplace=True)
```

```
df.columns
```

```
Index(['Brand', 'Price', 'Body', 'Mileage', 'EngineV', 'Engine Type',
      'Registration', 'Year', 'Price_log'],
      dtype='object')
```